

Clean Code: Meaningful Names (Full Summary)

1. Use Intention-Revealing Names

```
Bad:
int d;

Good:
int elapsedTimeInDays;
```

2. Avoid Disinformation

```
Bad:
accountList (not a list)

Good:
accounts
```

3. Make Meaningful Distinctions

```
Bad:
ProductData, ProductInfo

Good:
Product, ProductDetails
```

4. Use Pronounceable Names

```
Bad:
genymdhms

Good:
generationTimestamp
```

5. Use Searchable Names

```
Bad:
int x = 5;

Good:
int WORK_DAYS_PER_WEEK = 5;
```

6. Avoid Encoding

```
Bad:
String m_name;

Good:
String name;
```

7. Avoid Mental Mapping

```
Bad:
int c;

Good:
int customerCount;
```

8. Class Names

Use nouns. Avoid vague words like Manager, Data.

```
Good:
Customer, Order, AddressParser
```

9. Method Names

Use verbs.

Good:
save(), deleteUser(), getName(), isValid()

10. Don't Be Cute

Bad:
whack()

Good:
deleteUser()

11. Pick One Word per Concept

Bad:
get(), fetch(), retrieve()

Good:
use only 'get()' consistently

12. Avoid Pun (Same word ≠ different meaning)

Bad:
add() used for insert + math

Good:
add(), insert() separated

13. Use Solution Domain Names

Example:
JobQueue, CacheManager

14. Use Problem Domain Names

Example:
Account, Transaction, Customer

15. Add Meaningful Context

Bad:
state

Good:
address.state or Address class

16. Don't Add Unnecessary Context

Bad:
GSDAccountAddress

Good:
Address

Conclusion

Good naming makes code self-explanatory, easier to maintain, and improves collaboration.